Lecture 20 — Trees: Traversals, Diameter, Subtree Sizes, Tree DP, Rerooting

> Year Curriculum · Prerequisites: Week 14–15 (DP), Week 16 (BFS/DFS), Week 19 (spanning trees) · Estimated time: 3 hours

1. Overview

Trees are the most common special graphs in contests. They arrive as company hierarchies, file systems, road networks with exactly one route between cities, or as the output of an earlier algorithm (BFS trees, DFS trees, minimum spanning trees). Because a tree has no cycles, every pair of vertices is joined by exactly one path, and every vertex splits the tree into *independent* subtrees. That independence is what makes problems that are hard on general graphs — maximum matching, independent sets, longest paths — solvable in linear time on trees with dynamic programming.

This lecture covers:

- rooting a tree and computing parent, depth and subtree size,
- recursive and **iterative** traversals,
- the **diameter** (two-pass algorithm and DP),
- **tree DP** with the canonical examples of maximum matching and independent sets,
- an introduction to **rerooting**, which computes an answer for every possible root in O(n).

2. Trees and how they are given

For a graph with n vertices, the following are equivalent: (a) it is connected and acyclic; (b) it is connected and has n − 1 edges; (c) there is exactly one simple path between any two vertices. A *rooted* tree designates one vertex as the root; then each other vertex has a unique **parent** (the next vertex on its path to the root), **children**, a **depth** (distance from the root) and a **subtree** (itself plus all descendants).

Two input formats dominate:

1. **Edge list**: n − 1 lines `a b`. Build an undirected adjacency list and choose a root, conventionally vertex 1.
2. **Parent array**: for i = 2..n, the parent p_i. The tree is already rooted; you may build `children[p_i].push_back(i)` directly. Do not assume p_i < i unless the statement promises it.

Because there are no cycles, a traversal does not need a visited array: when exploring from u, skip only the vertex we came from (the parent). This is shorter and avoids bugs with stale visited flags across test cases.

3. Traversals

3.1 Recursive DFS

```
void dfs(int u, int p) {
    par[u] = p;
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;   // pre-order: information flowing down
        dfs(v, u);
        sz[u] += sz[v];            // post-order: information flowing up
    }
}
```

Two kinds of computation happen in a DFS:

- **Top-down (pre-order)**: values of a child depend on its parent — depth, distance from the root, the maximum value on the root-to-node path.
- **Bottom-up (post-order)**: values of a vertex depend on its children — subtree size, subtree sum, every tree DP in this lecture.

3.2 Avoiding deep recursion

A "bamboo" (a path) of $2 \cdot 10^5$ vertices recurses $2 \cdot 10^5$ levels deep. Codeforces and CSES configure a large stack and this is normally fine, but on judges with the default 8 MB (Linux) or 1 MB (Windows) stack it crashes. A robust alternative is to compute a **processing order** once:

```
vector<int> order, par(n + 1, 0);
vector<int> st = {1};
par[1] = -1;
while (!st.empty()) {
    int u = st.back(); st.pop_back();
    order.push_back(u);
    for (int v : adj[u]) if (v != par[u]) { par[v] = u; st.push_back(v); }
}
```

In `order`, every vertex appears after its parent. Iterating `order` from front to back performs top-down computations; iterating from back to front performs bottom-up computations (every child is processed before its parent). For example, subtree sizes:

```
vector<int> sz(n + 1, 1);
for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];
```

(BFS order works equally well.) All reference implementations below use this pattern, so they are safe on any judge.

3.3 Worked example — CSES 1674 *Subordinates*

Given the boss of each employee 2..n, output for each employee the number of (direct and indirect) subordinates. That is `sz[u] − 1`. With bosses `1 1 2 3`, the tree is 1 → {2, 3}, 2 → 4, 3 → 5; sizes are 5, 2, 2, 1, 1 and the answers `4 1 1 0 0`.

4. Diameter

The **diameter** is the longest path between two vertices (counted in edges, or in total weight for weighted trees).

4.1 Two passes

1. From an arbitrary vertex s, find a farthest vertex a.
2. From a, find a farthest vertex b.
3. dist(a, b) is the diameter.

Each pass is a BFS (unweighted) or DFS accumulating weights (non-negative weights), so the algorithm is O(n).

Why it works. Let (u, v) be any diameter, with tree path P between them, and let a be a farthest vertex from s. For a vertex z, call its *projection* the vertex of P closest to z; every path from z to a vertex of P passes through the projection. Let x be the projection of s and y the projection of a.

Since a is farthest from s, $d(s, a) \geq d(s, u) = d(s, x) + d(x, u)$. Also $d(s, a) \leq d(s, x) + d(x, a)$ (triangle inequality, valid for non-negative weights). Hence $d(x, a) \geq d(x, u)$, and likewise $d(x, a) \geq d(x, v)$.

- If y lies on the x…u part of P, then $d(x, a) = d(x, y) + d(y, a)$ and $d(x, u) = d(x, y) + d(y, u)$, so $d(y, a) \geq d(y, u)$. Therefore $d(a, v) = d(a, y) + d(y, v) \geq d(u, y) + d(y, v) = d(u, v)$.
- If y lies on the x…v part, the symmetric argument gives $d(a, u) \geq d(v, u)$.

Either way a is an endpoint of a path at least as long as the diameter, i.e. of a diameter, so the second pass from a finds the diameter. The argument uses non-negative weights; with negative weights the two-pass method can fail.

A useful corollary (CSES 1132 *Tree Distances I*): for every vertex v, the farthest vertex from v is one of the diameter endpoints a or b, so `max(dist_a[v], dist_b[v])` answers "maximum distance from v" for all v with three BFS passes.

4.2 Diameter by DP

Root the tree. Let `down[u]` be the length of the longest path starting at u and going down into its subtree. Then `down[u] = max over children c of (down[c] + w(u, c))`, 0 for a leaf. The longest path whose highest vertex is u combines the two largest values of `down[c] + w(u, c)` among u's children. The diameter is the maximum of that over all u.

This works for any weights (including negative ones, if you allow the path to stop early, i.e. take max with 0), and it is the template for tree DP: one value per vertex, computed from its children, answer combined at each vertex.

5. Tree DP

5.1 The recipe

1. **Root** the tree.
2. **State:** `dp[u][flag]` = optimal value (or count) *within the subtree of u*, where `flag` records the minimum information about u that the parent needs (is u selected? is u matched? what colour is u?).
3. **Transition:** combine the children's states. The subtrees of different children are disjoint and only interact through u, so the combination is usually a sum, product or max over children.
4. **Order:** post-order (children before the parent).
5. **Answer:** at the root, combined over its flags.

Compare with Week 14–15: the "prefix" becomes a "subtree", and "the previous element" becomes "the children".

5.2 Maximum matching — CSES 1130 *Tree Matching*

A matching is a set of edges with no shared endpoints. Maximise its size.

- `dp0[u]` = maximum matching in u's subtree in which u is **not** matched to one of its children.
- `dp1[u]` = maximum matching in u's subtree in which u **is** matched to one of its children.

If u is not matched downward, each child subtree is independent: `dp0[u] = Σ_c max(dp0[c], dp1[c])`. If u is matched to child c, then c must be unmatched within its subtree (contributing `dp0[c]`), we gain one edge, and every other child contributes its best. Writing `best(c) =`

`max(dp0[c], dp1[c])`:

```
dp1[u] = dp0[u] + max_c ( dp0[c] + 1 − best(c) )
```

For a leaf, `dp0 = 0` and `dp1` is impossible (we leave it 0, which is harmless here because `dp0` is also 0, but in general mark it as $-\infty$). The answer is `max(dp0[1], dp1[1])`. On the sample (edges 1-2, 1-3, 3-4, 3-5) the answer is 2, e.g. {1-2, 3-4}.

(There is also a greedy solution: repeatedly match a leaf with its parent. The DP generalises to weighted versions where greedy does not.)

5.3 Independent sets — AtCoder DP Contest P

Paint every vertex white or black so that no two adjacent vertices are both black; count the ways modulo 10^9+7.

```
white[u] = Π_c (white[c] + black[c])
black[u] = Π_c  white[c]
answer   = white[root] + black[root]
```

The flag is u's colour; children are independent given that colour, so the counts multiply. Maximum-weight independent set is the same DP with `+` instead of `×` and `max` instead of `+`.

5.4 Other classic tree DPs

- Subtree sums, counts of special vertices in each subtree.
- Minimum vertex cover (complement of maximum independent set on trees).
- "Choose k vertices in the subtree" knapsack on trees — $O(n^2)$ with the size-bounded merging trick; a regional-level topic.
- Distances: `down[u]` plus an `up[u]` computed top-down — which leads to rerooting.

6. Rerooting

Some problems ask for a value **for every choice of root**: the sum of distances from each vertex, the farthest vertex from each vertex, the best DP value if the tree were rooted at v. Running a separate O(n) DP for each root costs $O(n^2)$. Rerooting computes all answers in O(n) by:

1. computing the answer for one root (vertex 1) with an ordinary bottom-up DP, then
2. moving the root from a vertex p to a child c, deriving `ans[c]` from `ans[p]` in O(1) (or O(deg)), in top-down order.

6.1 Sum of distances — CSES 1133 *Tree Distances II*

For root 1, the sum of distances is simply Σ depth[v]. Now move the root from p to its child c. Every vertex in c's subtree (sz[c] of them) gets one step closer; every other vertex (n − sz[c] of them) gets one step farther. Therefore

```
ans[c] = ans[p] − sz[c] + (n − sz[c]) = ans[p] + n − 2·sz[c].
```

Process vertices in `order` (parents first) and apply the formula. On the sample tree (edges 1-2, 1-3, 3-4, 3-5): depths from 1 are 0, 1, 1, 2, 2 so ans[1] = 6; sz[3] = 3 gives ans[3] = 6 + 5 − 6 = 5; sz[2] = 1 gives ans[2] = 6 + 5 − 2 = 9; ans[4] = ans[5] = 5 + 5 − 2 = 8. Output `6 9 5 8 8`.

6.2 The general idea

In general, rerooting needs, for each vertex u and each neighbour c, the DP value of "the tree on u's side when the edge u–c is cut". For sums this is easy (subtract c's contribution). For max-type DPs you cannot "subtract" a max, so you keep prefix and suffix maxima over the children, or the top two values. CF 1092F *Tree with Maximum Cost* is a good next step (it follows the same `ans[c] = ans[p] + total − 2·sub[c]` shape with weights), and AtCoder DP *V — Subtree* is the full general version. Rerooting is revisited at a higher level in the World Finals track.

7. Reference implementations

All compile with `g++ -std=c++17 -O2 -Wall`. The diameter, matching and rerooting programs were additionally checked against brute force on random small trees.

7.1 Subtree sizes — CSES 1674 *Subordinates*

```cpp
#include <bits/stdc++.h>
using namespace std;
vector<vector<int>> children;
vector<int> sub;
void dfs(int u) {                          // post-order: children first
    sub[u] = 1;
    for (int c : children[u]) {
        dfs(c);
        sub[u] += sub[c];
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
```

```
    cin >> n;
    children.assign(n + 1, {});
    for (int i = 2; i <= n; i++) {
        int boss;
        cin >> boss;
        children[boss].push_back(i);
    }
    sub.assign(n + 1, 0);
    dfs(1);                                  // depth up to 2e5: fine on CSES/Codeforces
stacks
    for (int i = 1; i <= n; i++) cout << sub[i] - 1 << " ";
    cout << "\n";
}
```

7.2 Diameter by two BFS — CSES 1131 *Tree Diameter*

```
#include <bits/stdc++.h>
using namespace std;
int n;
vector<vector<int>> adj;
vector<int> bfs(int s) {                     // distances (in edges) from s
    vector<int> d(n + 1, -1);
    queue<int> q;
    d[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u])
            if (d[v] == -1) { d[v] = d[u] + 1; q.push(v); }
    }
    return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    adj.assign(n + 1, {});
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<int> d1 = bfs(1);
    int a = max_element(d1.begin() + 1, d1.end()) - d1.begin();   // farthest from 1
    vector<int> d2 = bfs(a);
    cout << *max_element(d2.begin() + 1, d2.end()) << "\n";       // farthest from a
}
```

7.3 Tree DP — CSES 1130 *Tree Matching*

```cpp
#include <bits/stdc++.h>
using namespace std;
// dp0[u] = best in subtree of u with u NOT matched to a child
// dp1[u] = best in subtree of u with u matched to one of its children
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    // iterative DFS order: parents appear before children
    vector<int> order, par(n + 1, 0);
    vector<int> st = {1};
    par[1] = -1;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        order.push_back(u);
        for (int v : adj[u])
            if (v != par[u]) { par[v] = u; st.push_back(v); }
    }
    vector<int> dp0(n + 1, 0), dp1(n + 1, 0);
    for (int i = n - 1; i >= 0; i--) {     // reverse order = children before parents
        int u = order[i];
        int bestGain = INT_MIN;            // gain from matching u with child c
        for (int c : adj[u]) {
            if (c == par[u]) continue;
            int bestC = max(dp0[c], dp1[c]);
            dp0[u] += bestC;
            bestGain = max(bestGain, dp0[c] + 1 - bestC);
        }
        if (bestGain != INT_MIN) dp1[u] = dp0[u] + bestGain;
    }
    cout << max(dp0[1], dp1[1]) << "\n";
}
```

7.4 Rerooting — CSES 1133 *Tree Distances II*

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
```

```
    vector<vector<int>> adj(n + 1);
    for (int i = 0; i < n - 1; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        adj[b].push_back(a);
    }
    vector<int> order, par(n + 1, 0), depth(n + 1, 0);
    vector<int> st = {1};
    par[1] = -1;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        order.push_back(u);
        for (int v : adj[u])
            if (v != par[u]) { par[v] = u; depth[v] = depth[u] + 1; st.push_back(v); }
    }
    vector<long long> sz(n + 1, 1), ans(n + 1, 0);
    for (int i = n - 1; i > 0; i--) sz[par[order[i]]] += sz[order[i]];   // subtree sizes
    for (int v = 1; v <= n; v++) ans[1] += depth[v];                      // answer at
root
    // moving the root from p to child c: sz[c] nodes get 1 closer, n - sz[c] get 1
farther
    for (int i = 1; i < n; i++) {
        int c = order[i];
        ans[c] = ans[par[c]] + n - 2 * sz[c];
    }
    for (int v = 1; v <= n; v++) cout << ans[v] << " ";
    cout << "\n";
}
```

8. Common mistakes

- **Parent check vs. visited array.** With an undirected adjacency list, forgetting `if (v == p) continue` makes DFS bounce between u and its parent forever.
- **n = 1.** There are no edges; make sure loops reading n − 1 edges and "farthest vertex" logic still work (diameter 0).
- **Stack overflow** on bamboo-shaped tests with recursive DFS on judges with small stacks; use the iterative order.
- **Two-pass diameter with negative weights** — use the DP.
- **Overflow.** The sum of distances can reach about $n^2/2 \approx 2 \cdot 10^{10}$ for $n = 2 \cdot 10^5$: `long long`.
- **Leaf states.** States that are impossible at a leaf (u matched to a child) must not be used as if valid; initialise to $-\infty$ when the DP could otherwise pick them.
- **Assuming p_i < i** in parent-array input when the statement does not guarantee it; then a simple reverse loop over i is not a valid bottom-up order.
- **Recomputing per root** ($O(n^2)$) when rerooting would give O(n).

9. Practice problems

- CSES 1674 — Subordinates — https://cses.fi/problemset/task/1674
- CSES 1131 — Tree Diameter — https://cses.fi/problemset/task/1131
- CSES 1132 — Tree Distances I — https://cses.fi/problemset/task/1132
- CSES 1133 — Tree Distances II — https://cses.fi/problemset/task/1133
- CSES 1130 — Tree Matching — https://cses.fi/problemset/task/1130
- AtCoder DP Contest P — Independent Set — https://atcoder.jp/contests/dp/tasks/dp_p
- AtCoder DP Contest V — Subtree — https://atcoder.jp/contests/dp/tasks/dp_v
- Codeforces 580C — Kefa and Park — https://codeforces.com/problemset/problem/580/C
- Codeforces 1092F — Tree with Maximum Cost — https://codeforces.com/problemset/problem/1092/F
- Codeforces 1528A — Parsa's Humongous Tree — https://codeforces.com/problemset/problem/1528/A

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 14 (Tree algorithms): traversal, diameter, all longest paths.
- USACO Guide, Gold: "Introduction to Tree Algorithms", "DP on Trees — Introduction" and "DP on Trees — Solving For All Roots" (https://usaco.guide/gold/).
- Looking ahead: Week 22 (LCA, binary lifting, Euler tour) answers path queries; the National track covers heavy-light and centroid decomposition.